HANDLING DATASET ERRORS AND OVERFITTING IN A CNN FOR WASTE CLASSIFICATION

A DEEP LEARNING FOR VISUAL RECOGNITION REPORT

Department of Computer Science
Aarhus University

Department of Computer Science
Aarhus University

Department of Computer Science
Aarhus University

December 5, 2025

ABSTRACT

This project investigates how different training strategies affect the performance of a Convolutional Neural Network for classifying household waste into 12 categories. The dataset contained duplicates, mislabeled samples, and class imbalance, which caused the initial model to achieve unrealistically high validation accuracy. Using FAISS, we identified and removed near duplicate images to produce a cleaner dataset and establish a reliable baseline. We then applied a series of methods to improve generalization, including normalization, data augmentation, weight decay, dropout, and learning rate schedules. The most effective improvements were obtained through normalization and augmentation and through weight decay, both reaching a validation accuracy of 82.5% compared to the corrected baseline of 78.1%. The experiments highlight the importance of dataset quality and regularization when training models on small and imperfect datasets.

1 Introduction

Effective waste recycling relies heavily on the correct sorting of waste materials, as different categories require specific recycling processes. To support and automate this sorting task, we constructed and trained a Convolutional Neural Network (CNN) using a public Kaggle dataset of approximately 15.150 images across 12 classes of household waste.

When training a CNN, you can encounter a lot of different obstacles along the way. Therefore, the main goal of this report is to investigate which methods improve the model and which do not. This includes looking into various training and architectural adjustments.

Furthermore, we will look at problems related to using an online dataset. Specifically, how can you work with a dataset that may contain errors or inconsistencies, and how can we alter the dataset to make it useful for training?

By addressing both model optimization and dataset integrity, this report aims to provide practical insights into developing a stable waste classification system.

2 Related Work

We originally believed that Kanani [2024] used a dataset similar to ours, which made their results appear comparable to ours. However, after looking more closely at both datasets, we realized that they are actually quite different. Their dataset is widely recognized as a strong benchmark for garbage classification, while ours is far less known and contains more issues. Their version is cleaner, better balanced, and has fewer problematic samples. This helps explain why their model performs more consistently, whereas our dataset introduced several challenges.

Another major difference is the number of classes. They work with six categories (cardboard, glass, paper, metal, trash, plastic), which is a very reasonable setup for waste sorting. In real systems, you rarely need more than these broad groups. For example, distinguishing between green and brown glass is usually unnecessary. Our dataset, on the other hand, contains 12 classes. This makes the classification task significantly harder, especially because some classes look very similar.

Although reducing the number of classes would likely have improved model performance and made the task more realistic, we chose not to modify the dataset, or pick another one. Instead, we kept the original dataset, with all 12 classes and focused on applying course methods to see how well we could improve results despite the dataset's limitations. This ended up being a valuable learning experience, showing how real-world datasets can be messy and how various preprocessing and training techniques can help deal with these challenges.

Several previous studies show that FAISS is an effective tool for detecting similar images in large datasets. This makes it relevant to our project, where we used FAISS to find and remove near-duplicate images from our own dataset.

Sun et al. [2021] demonstrate how FAISS can be used for fast and accurate image similarity search, even on very large datasets. Their method uses image embeddings and GPU-accelerated FAISS to detect duplicates and near-duplicates. This supports our approach, since we used FAISS for the same purpose: identifying images that are almost identical so they do not distort our training process.

Similarly, Rahman et al. [2024] use FAISS in a medical imaging setting to find subtle similarities between images. Their results show that FAISS works well even when image differences are very small. This is relevant to our project because some of the waste images in our dataset are also extremely similar.

3 Setup

We built and trained a Convolutional Neural Network (CNN) using a dataset from Kaggle[1] containing approximately 15.150 images across 12 classes of household waste:

Classes: Paper, Cardboard, Biological, Metal, Plastic, Green Glass, Brown Glass, White Glass, Clothes, Shoes, Batteries, and General Trash.

Each image in the dataset was already labelled, which allowed us to focus on designing and training the model rather than performing manual annotation.

Data Splitting: We divided the dataset into **80% training data** and **20% validation data**. This split ensures that we can evaluate whether the model learns general, meaningful patterns rather than simply memorising the training images.

Model Architecture: For our model we used PyTorch and applied *transfer learning* with a pretrained **ResNet-18**. This architecture is a common and reliable starting point for image-classification tasks, especially when working with datasets of this size.
We froze the early layers of the network so that the pretrained feature extractor remained unchanged, and trained only the final classification layer. We chose this setup because it represents a standard and effective approach when beginning with deep learning, and it helps reduce overfitting by relying on robust pretrained features.

Unless stated otherwise in later experiments, we kept most hyperparameters at their default or commonly used values:

Optimizer: Stochastic Gradient Descent, **Batch size:** 24, **Learning rate:** 0.001, **Scheduler:** None, **Epochs:** 60,

These choices served as our baseline configuration. We selected them because they are simple, standard, and well-suited for initial experiments in deep learning. This allowed us to establish a clear starting point before exploring more advanced improvements and tuning methods.

4 First issue: Dataset duplicates

We started out by running the model with above mentioned setup, but with only 20 epochs. When doing this, the model quickly reached a validation accuracy of 93%. This was of course not ideal, as it gave us little to work with. We tried

[1] https://www.kaggle.com/datasets/mostafaabla/garbage-classification

reducing the dataset down to 10% of the original size, but the results were still the same. This led us to believe that maybe our dataset was faulty. Meaning the same images, or very near duplicates, might appear multiple times, and therefore the same image could be in both the training and validation set, which obviously skews the results. This was the first problem we would seek out to try and fix.

4.1 Faulty dataset

To check if our presumption of the dataset being faulty was true, we used embeddings with FAISS. We did it pairwise, where the pairs were given a score from 0-1. By getting the top 5 most similar pairs, we could instantly see, that our assumptions were correct (See Figure 1).

Figure 1: Examples of the same images appearing twice

Not only was there multiple of the same image, we could also see, that some images were mislabeled (See Figure 2).

Figure 2: Wrong label example

To try and combat this, we used the FAISS scores to filter the dataset by removing the most alike images. We could set a threshold to do this. We experimented with this, until we found a validation accuracy that was low enough to be worked on. We ended up setting the threshold to 0.9, meaning a lot of the images were filtered out. This resulted in the dataset being 484 images, which we then did the 80/20 train/val split on. This meant the training set was 309 images and validation 97 images. This is what all future experiments are run with.

4.2 Result evaluation, debrief

This run produced a validation accuracy where the best accuracy were 78.16% and training accuracy were 98.78% as seen in Figure 3. Although the validation accuracy is lower than before, it is now more realistic and meaningful. Previously, the validation set accidentally contained images that were also present in the training set, which caused the validation accuracy to be misleadingly high. In other words, the model was being evaluated on data it had already seen, so the metric didn't reflect true generalization. Now that the validation set is properly separated from the training data, the validation accuracy actually measures how well the model performs on unseen images. Because of this, we can finally trust the validation accuracy as a reliable indicator of model performance.

Figure 3: Accuracy curves

5 Second issue: Overfitting

When looking at the graph (Figure 4), it became clear that the model is overfitting. Overfitting happens when a model learns the training data too well, including noise and details that do not generalize to unseen data. Overfitting is typically visible when, training loss keeps decreasing, while validation loss stops. Our graph showed exactly this pattern - indicating that the model started memorizing the training images instead of learning general features. This is also more normal for models with lesser images, as ours.

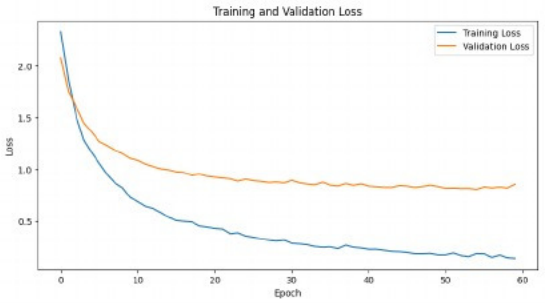

Figure 4: Loss curves

5.1 Tackling overfitting

There is a lot of different methods for tackling overfitting, so we tried to run experiments on which of these methods worked well on our model, and which didn't. Most of these experiments were variations of hyperparameter tuning.

Experiment 1: Normalization

To stabilize training and improve generalization, we applied input normalization as a preprocessing step. Neural networks train more effectively when input features are on similar scales; otherwise, large differences between pixel values can make weight updates inconsistent.

Normalization includes zero-centering, which means subtracting the mean so each input channel has an average of zero. Then scaling by the standard deviation to ensure all channels have similar spread. This prevents any single feature from dominating learning and helps the model converge faster.

We applied normalization and image augmentation simultaneously, so we only observed their combined effect. Ideally, each should be tested separately, but normalization is a standard preprocessing step in CNNs and is not primarily intended to reduce overfitting, making its inclusion reasonable.

Experiment 2: Augmentation

To combat overfitting, we applied data augmentation (such as random rotations, flips, color jitter, and random cropping) to artificially increase the diversity of the training data. Overfitting occurs when the model memorizes the specific training images instead of learning general patterns. This typically happens when the dataset is too small or not varied enough.
Data augmentation helps prevent this by creating new, slightly modified versions of the existing images each epoch. Even though the underlying label stays the same, the model sees the object under different conditions, angles, lighting, colors, or positions. This forces the network to learn more robust and general features rather than memorizing the exact pixel patterns of the training images.

Figure 5: Training and validation loss graph after introducing normalization and augmentation

Results on Experiments 1 and 2

As expected, the accuracy improved by about $\approx 3\%$, and the model showed better generalization. With this increased validation accuracy of $\approx 3\%$, we end up with a validation accuracy of 81.10%,. This is visible in Figure 5. The validation loss curve moves closer to the training loss curve, which is a good sign that the model is overfitting less.

We also noticed that the training loss curve became a bit more "messy," but this is normal when using data augmentation. Since the model sees slightly different versions of the images every epoch, the training becomes harder, but this actually

helps the model perform better on the validation set. This improvement is reflected in the $\approx 3\%$ increase in validation accuracy.

Experiment 3: Weight decay

Next, we experimented with weight decay (L2 regularization) to further reduce overfitting. Weight decay works by discouraging the model from assigning very large values to its weights. In other words, it "penalizes" overly strong connections in the network. This encourages the model to learn simpler, smoother patterns instead of memorizing the training data. By keeping the weights smaller, the network is less likely to overfit and is more likely to generalize well to new, unseen data. In practice, adding weight decay often results in more stable training and improved performance on the validation set.

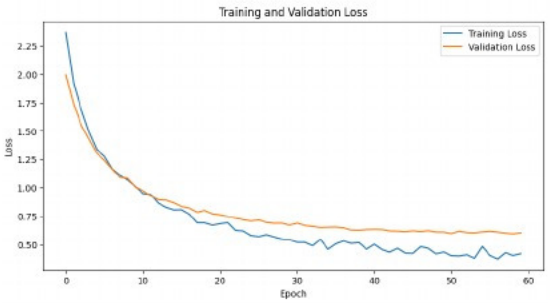

Figure 6: Training and validation loss graph after introducing weigh decay

Results: Weight decay

The improvement was modest but noticeable. The model became more stable, and the loss decreased as expected. As seen in Figure 6, the validation loss is now closer to the training loss, indicating better generalization. The best validation accuracy achieved was 82.5%.

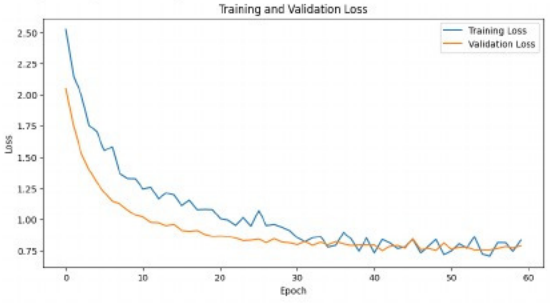

Figure 7: Training and validation loss graph after introducing dropout

Experiment 4: Dropout

Dropout is a simple yet effective regularization technique that helps reduce overfitting, especially in large networks where co adaptation of neurons can cause memorization of the training dataset. During training, dropout randomly removes a portion of the neurons in a layer, which prevents the model from relying on specific activations and encourages it to learn more robust features. This makes the network less sensitive to noise and improves its ability to generalize to new data, even though training can become slightly more difficult. We added dropout to our final layer to reduce overfitting by preventing co-adaptation between neurons.

Result: Dropout

Training loss decreased more slowly (as expected), but validation accuracy remained mostly unchanged but improved a bit (See Figure 7). So it did as excepted.

Experiment 5: Learning Rate Schedules

Learning rate (LR) is a key hyperparameter that controls how much the model's weights are updated during each training step. A higher learning rate allows the model to explore the loss landscape quickly, but may overshoot the optimal solution. A lower learning rate enables more precise convergence, but slows down learning.

For our fifth and final experiment, we tried out two different learning rate schedules:

1. Reducing the learning rate by a factor of 0.5 every 5 epochs.
2. Reducing the learning rate by a factor of 0.1 every 20 epochs.

Reducing the learning rate over time, is commonly used to help the model fine-tune its weights as training progresses. The idea is that large updates early in training help the model find a good region in the loss landscape, and smaller updates later allow it to converge more accurately to a minimum, potentially improving validation performance.

Results: Learning Rate Schedules

Applying these learning rate schedules made the performance worse (See Figure 8. The learning rate decreased before the model had enough time to learn the data, which limited its ability to find useful patterns. Validation accuracy did not improve, and training became less effective, leading to a higher final loss. Although the schedules reduced overfitting slightly, the loss in accuracy and training stability made them unsuitable for our setup.

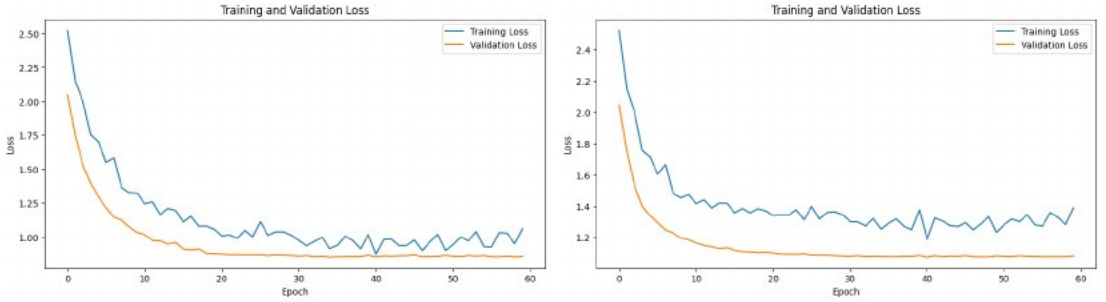

Figure 8: By a factor of 0.1 every 20 epochs (Left). By a factor of 0.5 every 5 epochs (Right)

Learning rate schedules can be very effective for larger datasets or deeper networks. However, in our smaller setup, reducing the learning rate too early hindered learning instead of improving it. This emphasizes the importance of adjusting learning rate schedules based on dataset size and model complexity. Since neither schedule provided a benefit, we reverted to the default setting: no learning rate schedule and a constant learning rate of 0.001.

6 Third issue: Class imbalance

One of the things we thought would be a bigger problem, was that the dataset has class imbalance, where the 'clothes' class has 34.64% of the total images. In raw numbers it means that 'clothes' has 187 images while 'plastic' for example only has 12 (See Table 1). This was another problem we wanted to look into.

Table 1: Image class distribution

Class Name	Amount	Percentage
battery	21	4.34%
biological	30	6.20%
brown-glass	21	4.34%
cardboard	41	8.47%
clothes	187	38.64%
green-glass	27	5.58%
metal	11	2.27%
paper	23	4.75%
plastic	32	6.61%
biological	53	10.95%
battery	26	5.37%
plastic	12	2.48%

We decided to take a closer look at our model's performance by examining the accuracy for each individual class, rather than just overall validation accuracy. This can be done using a confusion matrix or a normalized confusion matrix, which show how often the model predicts each class correctly and where it tends to make mistakes.

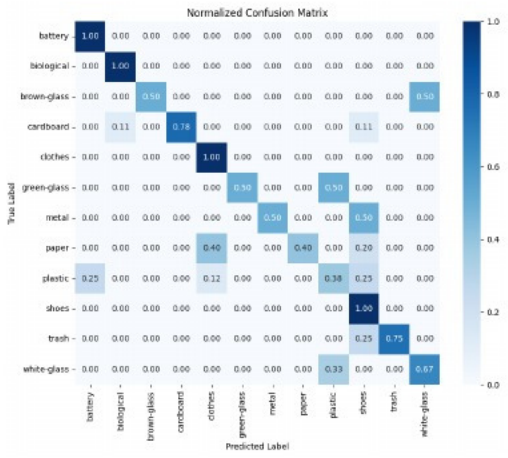

Figure 9: Normalized confusion matrix

By analyzing the normalized confusion matrix seen in Figure 9, we aimed to investigate whether class imbalance was causing the model to favor certain classes over others. For example, if the model sees many more images of clothes than brown glass, it might be more likely to predict "clothes" even when the input is plastic, because this guess is "safer" statistically. While this strategy can improve overall accuracy, it is harmful for the system's reliability and fairness, as it fails to recognize less frequent classes properly.

The matrix doesn't actually show this favoritism towards 'clothes' as much as expected. But it is still not very accurate in some classes. To solve this, multiple things could be done. We would have liked to add a sampler to the dataloader, so that each class was equally represented. This was however very slow, and we didn't end up having time to experiment with it. Another thing, we could have done, was to up-sample the lesser classes with augmented images or downsamle the bigger classes. These methods are not ideal, as they can introduce biases and information loss.

7 Results

Table 2: Comparison of Experiment Results

Experiment stage	Key adjustment	Training Acc	Validation Acc
Baseline (Initial run)	ResNet18, SGD, 20 epochs	95%	93%
Filtered baseline	FAISS filtered dataset (484 images), 60 epochs	99%	78.1%
Experiment 1	Augmentation and Normalization	94.1%	82.5%
Experiment 2	Weight Decay	94.2%	82.5%
Experiment 3	Dropout	81.9%	81.4%
Experiment 4	Scheduler - factor 0.1 every 20 epochs	64.3%	78.4%
Experiment 5	Scheduler - factor 0.5 every 5 epochs	71.6%	80.2%

The initial baseline model trained on the unfiltered dataset reached a validation accuracy of 93%, but this value was inflated due to duplicates and labeling errors. After filtering the dataset with FAISS, the model trained on the reduced dataset reached a validation accuracy of 78.1%. This serves as the corrected baseline for all later experiments.

Introducing normalization and augmentation improved generalization and increased validation accuracy to 82.5%. Adding weight decay produced the same validation accuracy, indicating that it stabilized training without altering overall performance. Adding dropout lowered training accuracy but maintained a similar validation accuracy of 81.4%. The two learning rate schedules resulted in reduced performance. The first schedule reached a validation accuracy of 78.4%, while the second reached 80.8%. Both schedules reduced overfitting but did not outperform the earlier improvements.

Across all experiments the best validation accuracy was 82.5%, achieved after adding normalization and augmentation and again after adding weight decay. These adjustments provided the most effective improvement relative to the corrected baseline. However, the dropout version reduced overfitting a lot, with the training accuracy being closer to the validation accuracy. Therefore, it could be argued that this version was the most ideal of our experiments.

7.1 Why these results?

Figure 10: Top (Right) and bottom (Left) 10 loss scores

We check the top and bottom 10 images in terms of loss in our model to see if there is a pattern in what our model guesses (See Figure 10).

We can see that there still is mislabeled images in the bottom 10, where it actually predicts correctly. This is of course very much not ideal. The model will have a ceiling, as it will guess correctly, but the label says it is incorrect. This will limit the the accuracy the model is able to achieve. This could also explain why the confusion matrix is skewed.

We can also see that the 'clothes' class is mostly represented in the top 10, which probably is because, that is the class the model sees the most during training. It also looks like the clothes is displayed the same way in all images, where the other images have more random angles.

8 Discussion

A stronger dataset, such as the one used in Kanani [2024], would likely have given us a more stable foundation and more realistic performance. Alternatively, if we wanted to build our own dataset, we could have taken the current images and manually cleaned them by removing duplicates, fixing incorrect labels, and discarding pictures that are not useful for garbage classification. This kind of careful dataset preparation would have made our results more trustworthy and reduced many of the challenges we encountered.

Our initial goal was to address overfitting in a small dataset, but the project gradually shifted toward a broader focus on dataset integrity. We developed an effective automatic method for detecting and removing duplicate images, and this worked well in practice. However, while this resolved the issue of duplication, it did not correct the mislabeled samples that remain in the dataset. These issues would require more careful manual cleaning, such as the process described earlier, where incorrect labels and unusable images could be identified and fixed. We explored several techniques to reduce overfitting, and together they produced improvements in generalization. Even so, the model would still benefit from further tuning and a more systematic approach to experimentation.

More experiments could have been done to further improve the model. First of all, we could have used a different optimization other than SGD. The ADAM model is widely known to be very reliant for most models, and it is something we could have looked in to. Furthermore, we could have tried to further alter the learning rate, to see if we had just hit a local minimum or if the model could go further. One thing we did not try is Cyclical Learning Rate (CLR). CLR is a method for setting the learning rate, which practically eliminates the need to experimentally find the best values and schedule for the global learning rates.

However, this was challenging in practice because we were learning many of the techniques at the same time as we implemented them. As we gained new knowledge about CNNs and training strategies, our understanding of the problems and possible solutions evolved. This meant our workflow was more exploratory and experimental than perfectly structured. Despite this, the process helped us learn a lot about diagnosing issues and applying the right tools to improve model performance. So now if we wanted to do this project again we would properly do it in a different way, and with a slightly different approach.

References

Jenil Kanani. Image recognition for garbage classification based on pixel distribution learning. *arXiv preprint arXiv:2409.03913*, 2024.

Xinlong Sun, Yangyang Qin, Xuyuan Xu, Guoping Gong, Yang Fang, and Yexin Wang. 3rd place: A global and local dual retrieval solution to facebook ai image similarity challenge. *arXiv preprint arXiv:2112.02373*, 2021.

MD Rahman, Feiroz Humayara, Syed Maudud E Rabbi, and Muhammad Mahbubur Rashid. Efficient medical image retrieval using densenet and faiss for birads classification. *arXiv preprint arXiv:2411.01473*, 2024.